

Backend 12

PHP Alapok

Rostagni Csaba

2025. július 29.

Tartalom

- 1 PHP Alapok
- 2 Operátorok
- 3 Változók és típusok
- 4 Vezérlési szerkezetek
- 5 Konzolos PHP

Tartalom

- 1 PHP Alapok
 - Bevezető
 - Megjegyzések
 - A PHP tag-ek

Mi az a PHP?

- Rekurzív betűszó (**PHP: H**ypertext **P**reprocessor)
„Personal Home Page Tools” névből származik
- Szkript nyelv
 - Értelmező szükséges hozzá
 - Hordozhatóbb, ahol van értelmező futnia kell
- Gyengén típusos
 - A 7-es verzióban már lehet típusokat megadni osztályokban
 - A 8-as verzióban már széles körben használhatóak a típusok

Hello World

hello-01.php

```
1 Hello World
```

Első PHP szkriptünk:

hello-02.php

```
1 <?php
2     echo "Hello World";
3 ?>
```

A kód tovább rövidítve:

hello-03.php

```
1 <?= "Hello World"; ?>
```

echo és var_dump()

```
$nev = "Bence";
```

PHP

```
echo $nev;
```

PHP

Eredmény

Bence

- Az `echo` megjeleníti a mögé írt kifejezéseket
- Mivel nem függvény, hanem nyelvi konstrukció, így a zárójelezés elhagyható

```
var_dump($nev);
```

PHP

Eredmény

string(5) "Bence"

- A megadott kifejezéseket jeleníti meg, a típusával együtt

Linkek:

- [echo - PHP Dokumentáció](#)
- [var_dump - PHP Dokumentáció](#)

Tartalom

- 1 PHP Alapok
 - Bevezető
 - **Megjegyzések**
 - A PHP tag-ek

Megjegyzések

PHP

```
<?php
// sor végéig érvényes megjegyzés

# sor végéig érvényes megjegyzés

/*
    akár több
    soros megjegyzés
*/
?>
```


Tartalom

- 1 PHP Alapok
 - Bevezető
 - Megjegyzések
 - A PHP tag-ek

Alapértelmezett PHP szintaxis

- Nyitó tag **<?php**
- Záró tag: **?>**
- A nyitó és záró tag közötti kódot a PHP értelmezőn keresztül fut
- Ami nincs közöttte az közvetlen a kimenetre íródik
- A záró tag benyeli az utána közvetlen következő sortörést

```
<?php
    $nev = "Luca";
    echo "Szia $nev ";
?>
jó újra látni.
```

PHP

Szia Luca jó újra látni.

Eredmény

Tartalom

- 1 PHP Alapok
- 2 Operátorok
- 3 Változók és típusok
- 4 Vezérlési szerkezetek
- 5 Konzolos PHP

Tartalom

2 Operátorok

- Összefűzés
- Aritmetikai operátorok
- Értékadó operátorok
- Logikai operátorok
- Short-circuit evaluation
- Összehasonlító operátorok
- Speciális operátorok

Összefűzés (Concatenation)

- Az összefűzés műveletét szövegeken alkalmazzuk, és a két szövegrész egymás után leírva kapjuk meg.
- PHP ban a "+" helyett a "." lesz a műveleti jel!**

```
echo "Szia" . "Peti";
```

PHP

Szia Peti

Eredmény

```
$nev = "Viki";  
echo "Szia" . $nev;
```

PHP

Szia Viki

Eredmény

```
echo "Szia" . $nev . " és Peti";
```

PHP

Szia Viki és Peti

Eredmény

Összefűzés (Concatenation) példa

<pre>echo 1 . 1;</pre>	PHP	Eredmény
		11

- Az 1-es szám mindkét esetben implicit átkonvertálódik szöveggé
- A két szöveg között összefűzés történik

<pre>echo 1.1;</pre>	Rossz példa!	Eredmény
		1.1

- Ez egy valós szám, nem összefűzés!

Tartalom

2 Operátorok

- Összefűzés
- **Aritmetikai operátorok**
- Értékadó operátorok
- Logikai operátorok
- Short-circuit evaluation
- Összehasonlító operátorok
- Speciális operátorok

Bináris aritmetikai műveletek

Jel	Művelet	Példa
+	Összeadás	5 + 3 // 8
-	Kivonás	8 - 2 // 6
*	Szorzás	8 * 2 // 6
/	Osztás	5 / 2 // 2.5
%	Maradék képzés	5 % 3 // 2
**	Hatványozás	2 ** 8 // 256

https:

[//www.php.net/manual/en/language.operators.arithmetic.php](https://www.php.net/manual/en/language.operators.arithmetic.php)

Tartalom

2 Operátorok

- Összefűzés
- Aritmetikai operátorok
- **Értékadó operátorok**
- Logikai operátorok
- Short-circuit evaluation
- Összehasonlító operátorok
- Speciális operátorok

Értékadó operátorok

Az egy darab egyenlőség jel (=) a klasszikus értékadó operátor.

Jel	Művelet	Példa	Eredmény
=	Értékadás	<code>\$a = 65</code>	<code>// 65</code>
+=	Összeadás és értékadás	<code>\$a += 5</code>	<code>// 70</code>
-=	Kivonás és értékadás	<code>\$a -= 60</code>	<code>// 10</code>
*=	Szorzás és értékadás	<code>\$a *= 5</code>	<code>// 50</code>
/=	Osztás és értékadás	<code>\$a /= 10</code>	<code>// 5</code>
.=	Összefűzés és értékadás	<code>\$a .= " db"</code>	<code>// 5 db</code>

https:

`//www.php.net/manual/en/language.operators.assignment.php`

Tartalom

2 Operátorok

- Összefűzés
- Aritmetikai operátorok
- Értékadó operátorok
- **Logikai operátorok**
- Short-circuit evaluation
- Összehasonlító operátorok
- Speciális operátorok

Logikai operátorok

- Precedencia szerint csökkenő sorrendbe találhatók a műveletek!
- A szöveges megadási forma alacsonyabb precedenciájú, mint az értékadás, így őket speciális esetben használjuk!
- A felsorolt operátorok rövidzár (short-circuit) kiértékelésűek.

Jel	Művelet	Példa
!	nem	! (1 > 2)
&&	és	1 > 2 && \$a > \$b
	vagy	1 > 2 \$a > \$b
AND	és	1 > 2 AND \$a > \$b
OR	vagy	1 > 2 OR \$a > \$b

Linkek:

- PHP dokumentáció: Logikai operátorok
- Sulinet logikai kifejezések

Tartalom

2 Operátorok

- Összefűzés
- Aritmetikai operátorok
- Értékadó operátorok
- Logikai operátorok
- **Short-circuit evaluation**
- Összehasonlító operátorok
- Speciális operátorok

Short-circuit evaluation

- Egy logikai kifejezés eredményéhez nem feltétlen kell teljesen kiértékelni
- Amennyiben egy "**ÉS**" kifejezés **bal oldalán hamis** érték áll, úgy a kifejezés értéke garantáltan **hamis** lesz.
- Amennyiben egy "**VAGY**" **bal oldalán igaz** érték áll, úgy a kifejezés értéke garantáltan **igaz** lesz.

Short-circuit evaluation példa (csak ÉS)

PHP

```
$a = 1 > 2;      // hamis  
$b = 5 != 6;     // igaz  
$c = 3 == 10;    // hamis  
$d = 1 <= 100;   // igaz  
$e = 100 >= 1;   // igaz  
  
var_dump($a && $b && $c && $d && $e);
```

Eredmény

```
bool(false)
```

- A `$a` értéke **hamis**
- Mivel minden változó között **ÉS** kapcsolat van, így már az az egy hamis az egész kifejezés értékét hamissá teszi
- A `$b`, `$c`, `$d` és `$e` értékeket nem vizsgálja

Short-circuit evaluation példa (csak VAGY)

PHP

```
$a = 1 > 2;      // hamis  
$b = 5 != 6;     // igaz  
$c = 3 == 10;    // hamis  
$d = 1 <= 100;   // igaz  
$e = 100 >= 1;   // igaz  
  
var_dump($a || $b || $c || $d || $e);
```

Eredmény

```
bool(true)
```

- A `$b` értéke **igaz**
- Mivel minden változó között **VAGY** kapcsolat van, így már az az egy igaz az egész kifejezés értékét igazzá teszi
- A `$c`, `$d` és `$e` értékeket nem vizsgálja

Short-circuit evaluation példa (ÉS, VAGY)

PHP

```
$a = 1 > 2;      // hamis  
$b = 5 != 6;     // igaz  
$c = 3 == 10;    // hamis  
  
var_dump($a && $c || $b && $c);
```

Eredmény

bool(false)

- `$a && $c` nem gyorsítható, értéke **hamis**
- A vagy jobb oldalát is ki kell értékelni
- A `$b && $c` nem gyorsítható, értéke **hamis**
- Végül a teljes kifejezést ki kell értékelni a végeredményhez

Tartalom

2 Operátorok

- Összefűzés
- Aritmetikai operátorok
- Értékadó operátorok
- Logikai operátorok
- Short-circuit evaluation
- **Összehasonlító operátorok**
- Speciális operátorok

Összehasonlító operátorok (egyenlőség)

Jel	Művelet	Példa
<code>==</code>	Egyenlő (csak az érték)	<code>5 == 5 // igaz</code>
<code>===</code>	Azonos (típus és érték)	<code>5 === "5" // hamis</code>
<code>!=</code>	Nem egyenlő (csak az érték)	<code>5 != 5 // hamis</code>
<code><></code>	Nem egyenlő (csak az érték)	<code>5 <> 5 // hamis</code>
<code>!==</code>	Nem azonos (vagy a típus vagy az érték)	<code>5 !== "5" // igaz</code>

Linkek:

- [Összehasonlító operátorok - PHP dokumentáció](#)

Összehasonlító operátorok

Jel	Művelet	Példa
<	Kisebb	1 < 2 // igaz
>	Nagyobb	1 > 2 // hamis
<=	Kisebb vagy egyenlő	1 <= 1 // igaz
>=	Nagyobb vagy egyenlő	5 >= 3 // igaz

Linkek:

- [Összehasonlító operátorok - PHP dokumentáció](#)

Tartalom

2 Operátorok

- Összefűzés
- Aritmetikai operátorok
- Értékadó operátorok
- Logikai operátorok
- Short-circuit evaluation
- Összehasonlító operátorok
- **Speciális operátorok**

Ternary operator (?:)

- A Ternary operátornak három operandusa van.
- Egy if else rövidített leírása.

`$eredmeny = $szam % 2 == 0 ? "páros" : "páratlan"`

feltétel "páros" "páratlan"
Érték, ha igaz Érték különben

```
$szam = 8;  
$eredmeny = $szam % 2 == 0 ? "páros" : "páratlan";  
echo $eredmeny;
```

PHP

páros

Eredmény

Ternary operator (?:) rövidítve

Amennyiben a feltétel és az igaz ág megegyezik utóbbi elhagyható.

```
$nev = "";  
$eredmeny = $nev ?: "ismeretlen";  
echo "Szia $eredmeny";
```

PHP

- A név üres , így logikailag hamis

Szia ismeretlen

Eredmény

```
$nev = "Robi";  
$eredmeny = $nev ?: "ismeretlen";  
echo "Szia $eredmeny";
```

PHP

- A névnek van konkrét értéke, így logikailag igaz

Szia Robi

Eredmény

Null Coalescing Operator (??)

`$eredmeny = $a ?? $b`

- Amennyiben a `$a` értéke `null`, akkor `$b` lesz az eredmény
- Különben az eredmény `$a` lesz.

```
$a = null;  
$b = 5;  
$eredmeny = $a ?? $b;  
  
echo $eredmeny; // 5
```

PHP

Null Coalescing Operator

- A Null Coalescing operátorok **egymásba ágyazhatóak**.
- Ilyenkor az első nem null érték lesz a végeredmény.

```
$a = null;  
$b = null;  
$c = "x";  
$d = "y";  
  
$eredmeny = $a ?? $b ?? $c ?? $d;  
  
echo $eredmeny; // x
```

PHP

Tartalom

- 1 PHP Alapok
- 2 Operátorok
- 3 Változók és típusok
- 4 Vezérlési szerkezetek
- 5 Konzolos PHP

Tartalom

- 3 Változók és típusok
 - Változók
 - Típusok áttekintése
 - Logikai értékek
 - Számok
 - Szövegek
 - Típusok kezelése

Változók elnevezése

- Minden változó **\$** jellel kezdődik!
- Tartalmazhat aláhúzást.
- Tartalmazhat számot.
- **Számmal nem kezdődhet!**
- Tartalmazhat kis- és nagybetűt is.
- **Érzékeny a kis- és nagybetűkre!**

Linkek:

- <https://www.php.net/manual/en/language.variables.basics.php>
- https://www.w3schools.com/php/php_variables.asp

Változó elnevezési példák

Helyes elnevezések:

```
$a = 1;  
$A = 2;  
$_a = 3;  
$a4 = 4;
```

PHP

Lehetséges, de inkább kerüljük az ékezetes betűk használatát!

Leggyakrabban a dollár jelet felejtik le.

Számmal nem kezdődhet a változó neve!

Nem tartalmazhat szóközt!

```
$á = 5;
```

PHP

```
a = 1;
```

Rossz példa!

```
$1a = 2;
```

Rossz példa!

```
$ _b _= _3;
```

Rossz példa!

Változók változó nevekben

Mivel minden változó **\$** jellel kezdődik, így akár a változó nevét is megadhatjuk egy változóval.

PHP

```
$x = "nev";  
$nev = "Sámuel";  
  
echo "$x";           // nev  
echo "$nev";         // Sámuel  
echo "$$x";          // $nev  
echo "${$x}";        // Sámuel  
echo $$x;            // Sámuel
```

$$\begin{array}{c} \$ \underbrace{\$x}_{\text{nev}} == \text{"Sámuel"} \\ \underbrace{\hspace{1.5cm}}_{\$nev} \end{array}$$

Tartalom

- 3 Változók és típusok
 - Változók
 - Típusok áttekintése
 - Logikai értékek
 - Számok
 - Szövegek
 - Típusok kezelése

Típusok

A PHP 10 alaptípust támogat

- Skaláris adattípusok
 - boolean
 - integer
 - float
 - string
- Összetett adattípusok
 - array
 - object
 - callable (meghívható függvények)
 - iterable (pszeudó típus, bejáró)
- Speciális típusok
 - resource (hivatkozás külső forrásra, pl.: fájl)
 - NULL

Linkek:

- [Típusok bevezető - PHP dokumentáció](#)

Tartalom

- 3 Változók és típusok
 - Változók
 - Típusok áttekintése
 - Logikai értékek
 - Számok
 - Szövegek
 - Típusok kezelése

Logikai értékek: a boolean típus

- Logikai értékeket tárolhatunk el benne.
- Csak igaz (**true**) vagy hamis (**false**) értékeket vehet fel.
- A megadása kis- és nagybetűkre **nem** érzékeny.

PHP

```
$b = true;  
$b = TRUE;  
$b = True;  
$b = TruE;  
  
$b = false;
```

Hamis értékek

- A logikai `false`
- A következő egész számok `0` és `-0`
- A következő valós számok `0.0` és `-0.0`
- Az üres szöveg `" "`
- Az egy darab 0-t tartalmazó szöveg: `"0"`
- Az üres tömb: `[]`
- A NULL típus: `null`
- A még be nem állított változók

Minden mást IGAZ-nak kell tekinteni.

Hamis értékek

PHP

```
<?php
var_dump((bool) "");           // bool(false)
var_dump((bool) 1);            // bool(true)
var_dump((bool) -2);           // bool(true)
var_dump((bool) "foo");        // bool(true)
var_dump((bool) 2.3e5);         // bool(true)
var_dump((bool) array(12));     // bool(true)
var_dump((bool) array());       // bool(false)
var_dump((bool) "false");       // bool(true)
?>
```

Tartalom

- 3 Változók és típusok
 - Változók
 - Típusok áttekintése
 - Logikai értékek
 - Számok
 - Szövegek
 - Típusok kezelése

Számok

- Egész számokat tárolhatunk el benne: `integer`
 - Mérete: `PHP_INT_SIZE` (rendszer függő)
 - 32-bites rendszeren kb. -2 milliárd...+2 milliárd
- A valós számokat lebegőpontosan tárolja: `float`
 - Mérete: rendszer függő
 - Tárolás módja:
IEEE Standard for Floating-Point Arithmetic (IEEE 754)

Linkek:

- Lebegőpontos számok pontatlansága

Tartalom

- 3 Változók és típusok
 - Változók
 - Típusok áttekintése
 - Logikai értékek
 - Számok
 - Szövegek
 - Típusok kezelése

Szöveg: a string típus

- A string típus egy byte-os karakterek sorozatát takarja.
- Így 256 különböző karaktert tud eltárolni.
- Nincs natív unicode támogatás
- A 32 bites rendszereken és PHP 7.0.0 előtt max méret: 2 GB
- Megadásának módjai
 - aposztróf
 - idézőjel
 - heredoc
 - nowdoc

Escaped characters

Minta	Röv.	Megnevezés	Jelentés
"\n"	LF	linefeed	soremelés
"\r"	CR	carriage return	kocsi vissza
"\t"	HT	horizontal tab	(vízszintes) tabulátor
"\e"	ESC	escape	
"\\ "	\	backslash	visszaperjel
"\\$"	\$	dollar sign	dollár
"\""	"	double-quote	idézőjel

A string literál megadása idézőjellel (")

A legegyszerűbb megadási forma:

```
$a = "Burger King";  
echo $a;
```

PHP

Burger King

Eredmény

```
echo "Subway";
```

PHP

Subway

Eredmény

Aposztróf esetén nincs szükség escape karakterre:

```
$b = "McDonald's";  
echo $b;
```

PHP

McDonald's

Eredmény

A string literál megadása idézőjellel (")

Az escape karakter nem csak az aposztróf jelhez használható, hanem sortöréshez is.

```
PHP  
echo "KFC\nTaco Bell";
```

Eredmény

```
KFC  
Taco Bell
```

A backslash karaktert itt már kell escapelni:

```
Rossz példa!  
echo "C:\tanar";
```

Eredmény

```
C:      anar
```

A változókat behelyettesíti:

```
PHP  
$w = "Wendy's";  
echo "A $w finom.";
```

Eredmény

```
A Wendy's finom.
```

A string literál megadása aposztróffal (')

A legegyszerűbb megadási forma:

```
$a = 'Burger King';  
echo $a;
```

PHP

Burger King

Eredmény

```
echo 'Subway';
```

PHP

Subway

Eredmény

Az escape karakter a "\" jel:

```
$b = 'McDonald\'s';  
echo $b;
```

PHP

McDonald's

Eredmény

A string literál megadása aposztróffal (')

Az escape karakter csak az aposztróf jelhez használható, **sortörésre már nem**.

```
echo 'KFC\nTaco Bell';
```

Rossz példa!

KFC\nTaco Bell

Eredmény

A backslash karaktert itt nem kell escapelni:

```
echo 'C:\tanar';
```

PHP

C:\tanar

Eredmény

Nem tud behelyettesíteni változót:

```
echo '$a';
```

Rossz példa!

\$a

Eredmény

Változók behelyettesítése egyszerű szintaxis

- Idézőjelek és heredoc használatakor a szövegbe behelyettesíti a változókat.
- Amint talál egy **\$** jelet úgy a lehető leghosszabb érvényes változónevet próbálja meg behelyettesíteni.

PHP

```
$nev = "Peti";  
echo "Hello $nev!";
```

Eredmény

Hello Peti!

Rossz példa!

```
$osztaly = "10.";   
echo "$osztalyA osztály";
```

Eredmény

Ismeretlen változó:
\$osztalyA

PHP

```
$a = 2;  
$b = "B";  
echo "$a$b osztály";
```

Eredmény

2B osztály

Változók behelyettesítése komplex szintaxis

- Idézőjelek és heredoc használatakor a szövegbe behelyettesíti a változókat.
- Tetszőleges változó, tömb, objektum tulajdonság behelyettesíthető, ha **kapcsos zárójelek közé tesszük**.

PHP

```
$nev = "Peti";  
echo "Hello {$nev}!";
```

Eredmény

Hello Peti!

PHP

```
$osztaly = "10.";   
echo "{$osztaly}A osztály";
```

Eredmény

10.A osztály

PHP

```
$t = ["A", "B", "C"];  
echo "1:{$t[0]}\n2:{$t[1]}";
```

Eredmény

1:A
2:B

Hozzáférés a szöveg karaktereihez

- A string típusra gondolhatunk úgy, mint egy 0-tól indexelt tömbre.
- A szögletes zárójelekkel tetszőleges karaktert elérhetünk.
- A (PHP 7.1.0) negatív értékeket is elfogadja, ekkor hátulról indul az indexelés .

PHP

```
$nev = "Laci";  
echo "{$nev[0]}\n";  
echo "{$nev[1]}\n";  
echo "{$nev[-1]}\n";
```

Eredmény

L
a
i

A multibyte-os karakterek esetén az eredmény értelmezhetetlen lehet:

Rossz példa!

```
$nev = "László";  
echo "{$nev[0]}\n";  
echo "{$nev[1]}\n";  
echo "{$nev[-1]}\n";
```

Eredmény

L
?
?

Tartalom

- 3 Változók és típusok
 - Változók
 - Típusok áttekintése
 - Logikai értékek
 - Számok
 - Szövegek
 - Típusok kezelése

Típus vizsgálat

Az alábbi függvények logikai értéket adnak vissza.

fv	vizsgálat
<code>is_array()</code>	Tömb
<code>is_bool()</code>	Logikai
<code>is_callable()</code>	Meghívható-e, mint függvény
<code>is_float()</code>	Lebegőpontos szám
<code>is_int()</code>	Egész szám
<code>is_null()</code>	Null érték
<code>is_numeric()</code>	Szám, vagy szöveggént tárolt szám
<code>is_object()</code>	Objektum
<code>is_scalar()</code>	integer, float, string vagy boolean
<code>is_string()</code>	Szöveg

Érték kiolvasása típus szerint

Az alábbi függvények a változóból kinyerik a megadott típusú értéket.

fv	típus
<code>boolval()</code>	Logikai
<code>doubleval()</code>	Valós szám
<code>floatval()</code>	Valós szám
<code>intval()</code>	Egész szám
<code>strval()</code>	Szöveg

floatval()

- Amennyiben egy szöveg számmal kezdődik, úgy annak az értékét adja vissza.
- Egyéb esetben 0-t ad vissza.
- Nulla esetén érdemes ellenőrizni, hogy nem e nulla volt az eredeti szöveg.

```
$s = "10.5 kg";  
$mennyiseg = floatval($s);  
echo $mennyiseg;
```

PHP

10.5

Eredmény

Tartalom

- 1 PHP Alapok
- 2 Operátorok
- 3 Változók és típusok
- 4 Vezérlési szerkezetek
- 5 Konzolos PHP

Tartalom

- 4 Vezérlési szerkezetek
 - Elágazások

Elágazás: if

Egyszerű elágazás:

```
<?php
if ( $x > 10 )
{
    echo "$x nagyobb, mint 10.";
}
```

PHP

Elágazás: if ... else

PHP

```
<?php
$szam = 10;
if( $szam % 2 == 0 )
{
    echo "A szám ({ $szam }) páros.";
}
else
{
    echo "A szám ({ $szam }) páratlan.";
}
```

Eredmény

A szám (10) páros.

Elágazás: if ... elseif

PHP

```
<?php
$a = 1; $b = 2;
if( $a > $b )
{
    echo "A nagyobb, mint B";
}
elseif ( $a < $b )
{
    echo "B nagyobb, mint A";
}
else
{
    echo "A és B egyenlőek."
}
```

Többirányú elágazás: switch

PHP

```
<?php
switch ($változo) {
    case "alma":
        echo "A változó értéke: alma";
        break;
    case 1:
    case 2:
        echo "A változó értéke egy szám";
        break;
    default:
        echo "Minden más esetben ide jut.";
        break;
}
```

Tartalom

- 1 PHP Alapok
- 2 Operátorok
- 3 Változók és típusok
- 4 Vezérlési szerkezetek
- 5 Konzolos PHP

Tartalom

- 5 Konzolos PHP
 - Konzolos szkript használata
 - A szkript argumentumai

PHP konzolos verzió: php-cli

- PHP-ban lehetséges konzolos alkalmazások készítése is. Ezeket a **php-cli**, azaz a **PHP Command Line Interface** segítségével futtathatjuk.

Teszteljük a konzolos PHP verzióját.

```
php -v
```

CMD

Súgó megjelenítése

```
php -h
```

CMD

Interaktív mód

```
php -a
```

CMD

Tartalom

- 5 Konzolos PHP
 - Konzolos szkript használata
 - A szkript argumentumai

Konzolos szkript paraméterezése

- A legtöbb program indításakor paramétereket adhatunk meg, hogy speciálisan indíthassuk el, például a böngészőt rögtön privát módban.
- PHP-ban, ha nincs letiltva erre jó az `$argv` és az `$argc`
- Az `$argv` (argumentum values) a paraméterül kapott értékek (azaz az argumentumok) tömbjét jelenti
- Az `$argc` (argument count) egy olyan szám, ami megmondja, hogy hány argumentumot kapott.
- A tömb minden eleme string típusú lesz, még a számok is!

Konzolos szkript paraméterezése példa

Futtassuk a hello.php-t a megadott módon.

```
php hello.php a 10 "x y"
```

CMD

```
<?php
var_dump($argv);
var_dump($argc);
```

hello.php

```
array(4) {
    [0]=> string(8) "hello.php"
    [1]=> string(1) "a"
    [2]=> string(2) "10"      // szöveggént tárolt szám!
    [3]=> string(3) "x y"
}
int(4)
```

Eredmény